Методические указания к лабораторной работе

Автор: доцент НИТПУ Друки А.А.

Семантическая сегментация изображений

Данная лабораторная работа посвящена семантической сегментации изображений кошек и собак с помощью сверточного автоэнкодера.

Цель работы: получить навыки семантической сегментации изображений кошек и собак на два класса с помощью сверточного автоэнкодера.

Задачи:

1. Ознакомиться с работой сверточных автоэнкодеров в библиотеке Keras;
2. Научиться решать задачу сегментации изображений кошек и собак на два класса.

Сегментация изображений

Сегментация изображения – разбиение изображения на непересекающиеся области, каждая из которых представляет некоторую категорию (класс). Чтобы сегментировать изображение, в глубоком обучении используется модель для назначения класса каждому пикселю изображения – таким образом изображение сегментируется на разные зоны (например «фон» и «кошка» или «трава», «небо» и «деревья») (рисунок 1). Данная категория методов может найти применение в инструментах редактирования изображений и видео, автоматическом управлении транспортными средствами, робототехнике, медицине и т. д.

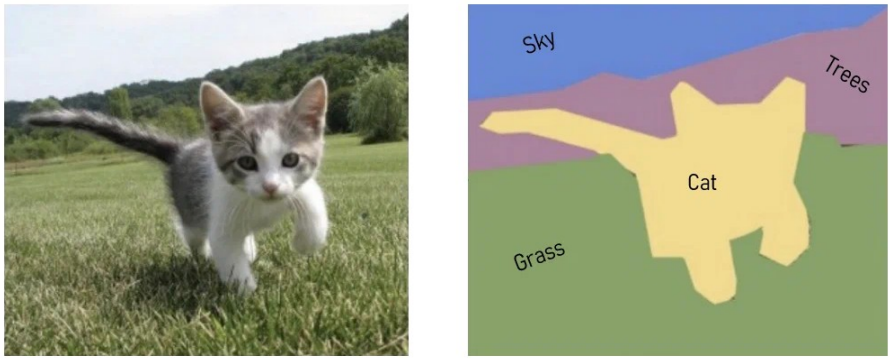


Рисунок 1. Пример сегментации изображения

Есть два разных варианта сегментации изображений:

Семантическая сегментация – когда каждый пиксель независимо друг от друга относится к некоторой семантической категории (скажем, «кошка»). В этом случае, если на изображении есть две кошки, все соответствующие пиксели будут включены в одну и ту же общую категорию – «кошка».

Сегментация экземпляров – направлена не только на классификацию пикселей изображения по категориям, но и на выделение отдельных экземпляров объекта. Поэтому на изображении с двумя кошками экземпляры «кошка 1» и «кошка 2» будут распознаны как две отдельные группы пикселей.

В данной работе мы сосредоточимся на семантической сегментации: в примере ниже мы исследуем изображения кошек и собак и постараемся научиться различать основной предмет и его фон.

Обучающая выборка

Для работы нам понадобится набор данных **Oxford-IIIT Pets** (www.robots.ox.ac.uk/ ~vgg/data/pets/), содержащий 7390 изображений различных пород кошек и собак вместе с соответствующими масками сегментации (рисунок 2). Маска сегментации – это эквивалент метки в задаче сегментации изображения; изображение того же размера, что и входное изображение, с одним цветовым каналом, в котором каждое целочисленное значение обозначает класс соответствующего пикселя на входном изображении. В нашем случае пиксели масок сегментации могут принимать одно из трех целочисленных значений.

1 (объект);

2 (фон);

3 (контур).

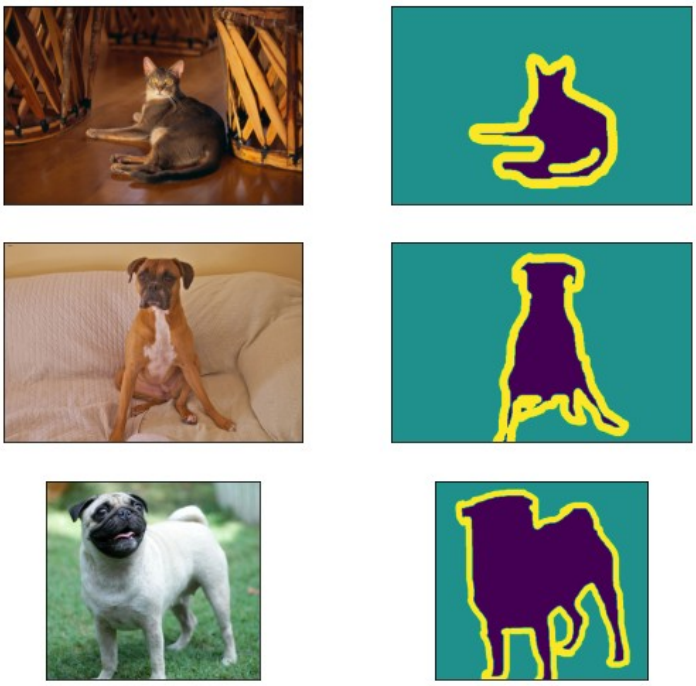


Рисунок 2. Примеры изображений из набора Oxford-IIIT Pets

Ход работы

1. Загрузка данных

Для начала загрузим и распакуем набор данных, использовав утилиты wget и tar:

```
!wget http://www.robots.ox.ac.uk/~vgg/data/pets/data/images.tar.gz
!wget http://www.robots.ox.ac.uk/~vgg/data/pets/data/annotations.tar.gz
!tar -xf images.tar.gz
!tar -xf annotations.tar.gz
```

Входные изображения сохраняются в виде файлов JPG в папке images/ (например, images/Abyssinian_1.jpg), а соответствующие маски сегментации – в виде файлов PNG с теми же именами в папке annotations/trimaps/ (например, annotations/trimaps/Abyssinian_1.png).

Импортируем необходимые библиотеки для работы:

```
import numpy as np
import random
import matplotlib.pyplot as plt
from tensorflow.keras.utils import load_img, img_to_array
import os
```

2. Предобработка данных

Подготовим список путей к входным файлам и файлам масок:

```
input_dir = "images/"
target_dir = "annotations/trimaps/"

input_img_paths = sorted([os.path.join(input_dir, fname)
                  for fname in os.listdir(input_dir)
                  if fname.endswith(".jpg")])

target_paths = sorted([os.path.join(target_dir, fname)
                  for fname in os.listdir(target_dir)
                  if fname.endswith(".png") and not fname.startswith(".")])
```

Для того чтобы проверить, правильность загрузки наших данных выведем несколько загруженных изображений и их соответствующие маски (рисунок 3).

```
num_images = 6
random_indices = random.sample(range(len(input_img_paths)), num_images)

plt.figure(figsize=(15, 5))

for i, idx in enumerate(random_indices):
    plt.subplot(2, num_images, i + 1)
    plt.imshow(load_img(input_img_paths[idx]))
    plt.title(f"Изображение {idx}")
    plt.axis('off')

    plt.subplot(2, num_images, num_images + i + 1)
    mask = img_to_array(load_img(target_paths[idx]))
    plt.imshow(mask[:, :, 0])
    plt.title(f"Маска {idx}")
    plt.axis('off')

plt.tight_layout()
plt.show()
```



Рисунок 3. Вывод исходных изображений на экран

Далее загрузим входные данные и цели в два массива NumPy и разделим массивы на обучающий и проверочный. Так как набор данных невелик, его можно целиком загрузить в память.

Все изображения будут масштабироваться до размеров 200 × 200:

```
img_size = (200, 200)
```

Общее количество образцов:

```
num_imgs = len(input_img_paths)
```

Перемешивание файлов (изначально они были отсортированы по породам). В обеих инструкциях используется одно и то же начальное число (1337), чтобы гарантировать единый порядок входных и целевых файлов.

```
random.Random(1337).shuffle(input_img_paths)
random.Random(1337).shuffle(target_paths)
```

```
def path_to_input_image(path):
    return img_to_array(load_img(path, target_size=img_size))

def path_to_target(path):
    img = img_to_array( load_img(path, target_size=img_size,
color_mode="grayscale"))
    img = img.astype("uint8") - 1
    return img
```

Загрузка всех изображений в массив **input_imgs** типа **float32** и масок в массив **targets** типа **uint8** (в одном и том же порядке). Входные изображения имеют три канала (значения RBG), а цели – один канал (с целочисленными метками).

```
input_imgs = np.zeros((num_imgs,) + img_size + (3,), dtype="float32")
targets = np.zeros((num_imgs,) + img_size + (1,), dtype="uint8")
for i in range(num_imgs):
    input_imgs[i] = path_to_input_image(input_img_paths[i])
    targets[i] = path_to_target(target_paths[i])
```

Зарезервируем 1000 образцов для проверочного набора:

```
num_val_samples = 1000
```

Разделим данные на обучающий и проверочный наборы, содержащие исходные изображения и сегментированные маски:

```
.
.
Допишите код
.
.
```

3. Реализация топологии нейронной сети

Для сегментации изображений мы будем использовать нейронную сеть класса автоэнкодер. С работой автоэнкодеров вы познакомились в прошлых примерах.

Первая половина модели очень похожа на типичную сверточную сеть, используемую для классификации изображений: стек слоев Conv2D с постепенно увеличивающимся количеством фильтров. Цель этой первой половины – закодировать изображения в карты признаков меньшего размера, где каждое пространственное местоположение (или пиксель) содержит информацию о большем пространственном фрагменте исходного изображения. Конечный эффект можно интерпретировать как сжатие.

Попробуйте реализовать вторую половину нейронной сети как стек слоев Conv2DTranspose. Данный тип слоев можно рассматривать как сверточный слой, который увеличивает разрешение. Таким образом этот слой является обратной операцией свертки.

Нейронную сеть можно реализовать в виде функции, которую можно в последствии вызывать в любом удобном месте.

```
from tensorflow import keras
from tensorflow.keras import layers


.
.
Допишите код
.
.
```

Conv2D – создание сверточного слоя со следующими параметрами: количество плоскостей (фильтров), размер ядра свертки, **activation** – функция активации, **padding** – параметр "same" означает не изменять размер изображения после его обработки сверточным слоем.

Conv2DTranspose – создание слоя с обратной функцией свертки. Другими словами, это слой развертки.

Определим модель нейронной сети:

```
.
.
Допишите код
.
.
```

Теперь скомпилируем и обучим модель:

```
.
```

```
.
Допишите код
.
.
```

При компиляции модели мы выбрали следующие параметры: алгоритм обучения **RMSPRop**, функцию ошибки **sparse_categorical_crossentropy**, которая хорошо подходит для задач сегментации.

4. Обучение нейронной сети

Для обучения нейронной сети необходимо вызвать метод **fit()**:

```
.
.
Допишите код
.
.
```

train_input_imgs – входные данные.
train_targets – целевые данные (сегментированные маски).
epochs – количество повторений циклов обучения для всей выборки данных.
callbacks – обратные вызовы.
batch_size – размер пакета данных который будет пропускаться через нейронную сеть, и только после этого будут изменяться веса модели.
validation_data – валидационная выборка.

Построим графики изменения ошибки на этапах обучения и проверки (рисунок 4):

```
epochs = range(1, len(history.history["loss"]) + 1)
loss = history.history["loss"]
val_loss = history.history["val_loss"]
plt.figure()
plt.plot(epochs, loss, "bo", label="Потери на этапе обучения")
plt.plot(epochs, val_loss, "b", label="Потери на этапе проверки")
plt.title("Потери на этапах обучения и проверки")
plt.legend()
```

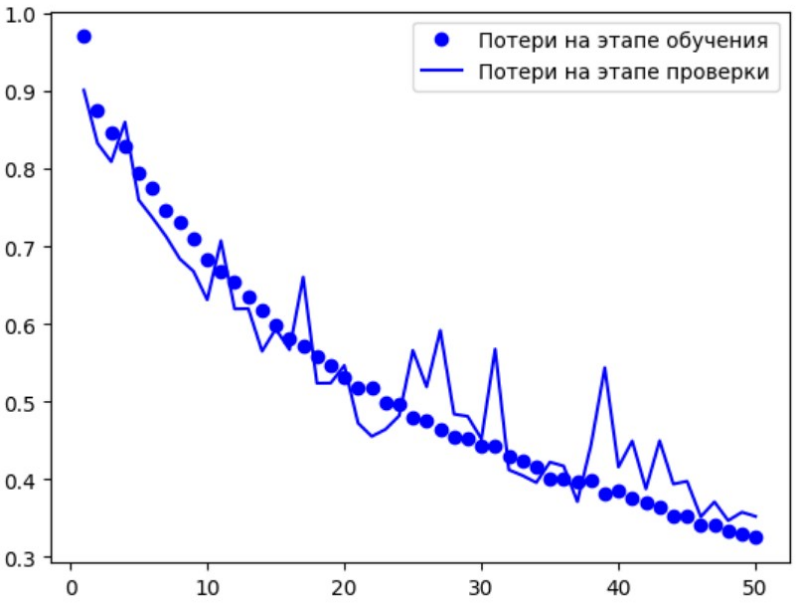


Рисунок 4. График изменения ошибки в процессе обучения и проверки

Как можно заметить на рисунке 5, на полпути (где-то в районе 25-й эпохи) начал проявляться эффект переобучения.

5. Результат работы нейронной сети

Выведем несколько случайных изображений из тестовой выборки и результат их сегментации.

```
random_indices = random.sample(range(len(val_input_imgs)), 4)
fig, axes = plt.subplots(3, 4, figsize=(11, 7))

for i, idx in enumerate(random_indices):
    axes[0, i].imshow(np.array(val_input_imgs[idx], dtype='uint8'))
    axes[0, i].set_title(f"Изображение {idx}")
    axes[0, i].axis('off')

for i, idx in enumerate(random_indices):
    axes[1, i].imshow(np.array(val_targets[idx], dtype='uint8'))
    axes[1, i].set_title(f"Маска {idx}")
    axes[1, i].axis('off')

predictions = model.predict(np.array([val_input_imgs[idx] for idx in
random_indices]))
for i, idx in enumerate(random_indices):
    axes[2, i].imshow(np.array(np.argmax(predictions[i], axis=-1),
dtype='uint8'))
    axes[2, i].set_title(f"Результат {idx}")
    axes[2, i].axis('off')

plt.tight_layout()
plt.show()
```

На рисунке 5 представлен пример исходного изображения, целевой семантической маски и полученного результата. В получившихся масках есть

небольшие артефакты, обусловленные присутствием геометрических фигур на переднем и заднем планах, но в целом модель дала довольно точный результат.

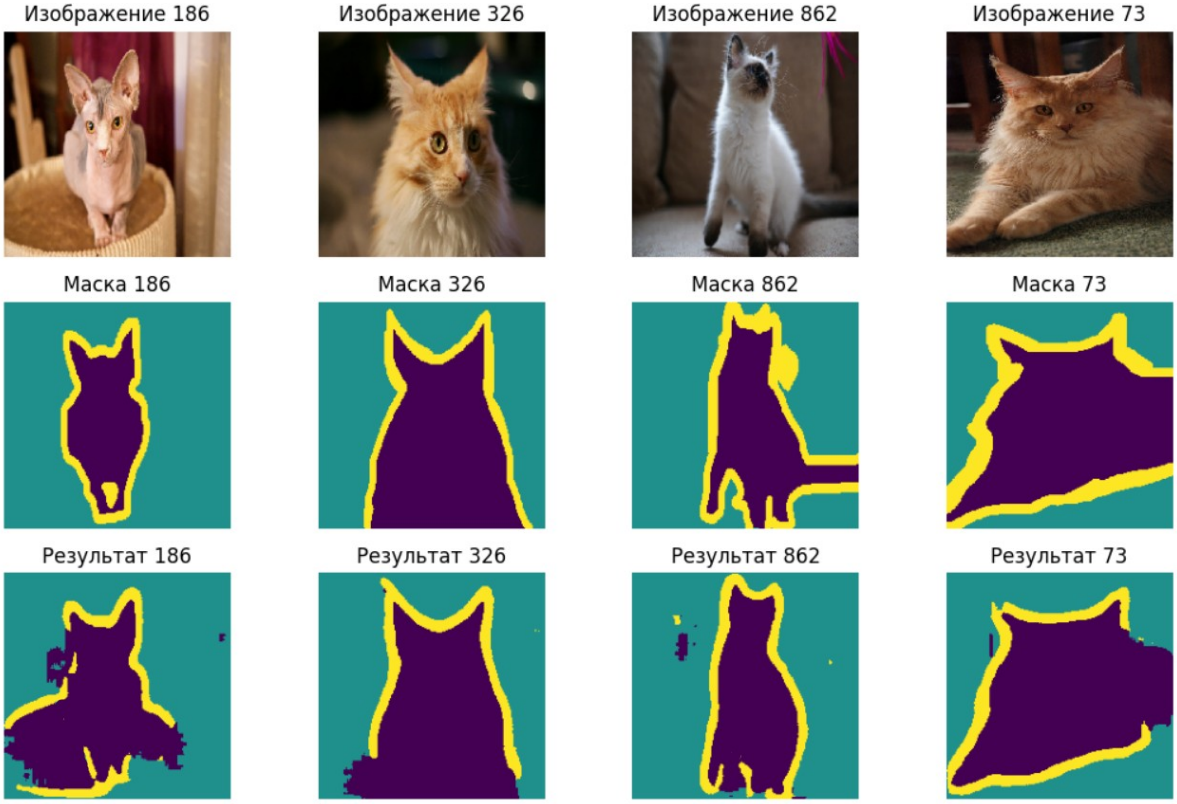


Рисунок 5. Пример сегментации изображения.

Вывод

В результате выполнения данной лабораторной работы Вы получили навыки семантической сегментации изображений кошек и собак на два класса с помощью сверточного автоэнкодера.

Критерии оценки

Максимальное число баллов, которое можно получить по лабораторной работе – 8 баллов.

Минимальное число баллов, при котором задание засчитывается – 5 баллов.

Выполнение задания

Выполнено меньше половины задания – 0 баллов.

Большей частью выполнено – 2 балл.

Выполнено полностью – 4 балла.

Качество отчета

Сильные огрехи в оформлении или удовлетворяет менее половины требований к отчету – 0 баллов.

Хорошее оформление. Удовлетворяет большей части требований – 2 балл.

Соответствует всем требованиям – 4 балла.